

StrayCare: A Community-Driven Platform for Animal Rescue and Adoption

BIB04 Sanath Bhat BIB21 Karttikey Chavali
BIB23 Hariom Khaladkar BIB26 Rajat Kulkarni

2025 – 2026

Contents

Acknowledgement	7
Abstract	8
Chapter 1: Introduction to Project Topic	9
1.1 Overview	9
1.1.1 Project Overview	9
1.2 Brief Description	9
1.3 Problem Definition	10
1.4 Applying Software Engineering Approach	10
Chapter 2: Literature Survey	12
2.1 Incident Reporting and Alert Systems	12
2.2 Case Management and Citizen Tracking	12
2.3 Adoption and Rehoming Processes	13
2.4 NGO Verification and Accountability	13
2.5 Fragmented Systems	13
Chapter 3: Software Requirements Specification	17
3.1 Introduction	17
3.1.1 Purpose	17
3.1.2 Intended Audience and Reading Suggestion	17
3.1.3 Project Scope	17
3.1.4 Design and Implementation Constraints	18
3.1.5 Assumption and dependencies	18
3.2 System Feature	18
3.2.1 User Authentication and Authorization	18
3.2.2 Case Management and Workflow System	19
3.3 External Interface Requirements	19

3.3.1 User Interface	19
3.3.2 Software Interface	20
3.3.4 Communication Interface	20
3.4 Non-Functional Requirements	20
3.4.1 Performance Requirements	20
3.4.2 Safety Requirements	21
3.4.3 Security Requirements	21
3.4.4 Software Quality Attributes	21
3.5 Analysis Model	22
Chapter 4: System Design	23
4.1 System Architecture	23
4.2 UML Diagrams	24
4.2.1 Module 1: Case Reporting & Management	25
4.2.2 Module 2: Pet Adoption Workflow	25
4.2.3 Module 3: NGO Verification & Feedback	26
4.2.4 Class Diagram (Fig 4.11)	26
4.2.5 Object Diagram (Fig 4.12)	27
4.2.6 Component Diagram (Fig 4.13)	27
4.2.7 Deployment Diagram (Fig 4.14)	27
Chapter 5: Literature Survey on “Rule-Based Approach in Natural Language Processing (NLP)”	28
5.1 Introduction	28
5.2 Previous Research	28
5.3 State of Art Algorithms	28
5.4 Handcrafted Rules in Rule-Based Sentiment & Severity Analysis	28
Chapter 6: Machine Learning Concepts and Implementation	30
6.1 Methodology and Algorithms	30
6.2 Confusion Matrix for Decision Tree Classifier	31
6.3 Enhancements and Future Work	31
Chapter 7: Software Testing and Quality Assurance	32
7.1 Introduction to Testing	32
7.2 Automated Testing	32
7.3 Manual Testing	33
Chapter 8: Applications of Rule Based Approach in NLP systems	36
Chapter 9: Methodology and Results	37
9.1 Technology Details used in the project	37
9.2 Procedure of the project	38
9.3 Results	39
9.3.1 Citizen Module	39
9.3.2 NGO Module	40

9.3.3 Admin Module	41
Chapter 10: Conclusion	42
Chapter 11: References	43
Chapter 12: Project Plan 1.0	45

A PROJECT REPORT ON

"StrayCare: A Community-Driven Platform for Animal Rescue and Adoption"

SUBMITTED BY

BIB04 Sanath Bhat
BIB21 Karttikey Chavali
BIB23 Hariom Khaladkar
BIB26 Rajat Kulkarni

GUIDED BY

Dr. Bhavana Kanawade

**DEPARTMENT OF INFORMATION TECHNOLOGY
INTERNATIONAL INSTITUTE OF INFORMATION
TECHNOLOGY, PUNE - 411057
SAVITRIBAI PHULE PUNE UNIVERSITY
2025 – 2026**

INTERNATIONAL INSTITUTE OF INFORMATION TECHNOLOGY
HINJEWADI, PUNE-57

DEPARTMENT OF INFORMATION TECHNOLOGY

Certificate

This is to certify that the project report entitled "**StrayCare: A Community-Driven Platform for Animal Rescue and Adoption**" which is being submitted by:

- **BIB04 Sanath Bhat**
- **BIB21 Karttikey Chavali**
- **BIB23 Hariom Khaladkar**
- **BIB26 Rajat Kulkarni**

have partially completed the Project entitled "**StrayCare: A Community-Driven Platform for Animal Rescue and Adoption**", under my guidance in partial fulfillment of the requirement for the award of the Bachelors of Engineering in Information Technology of International Institute of Information Technology, Hinjawadi, by Savitribai Phule Pune University for the academic year 2025 – 2026.

Dr. Bhavana Kanawade
Guide

Internal Examiner: _____

Dr. Jyoti Surve
Head of the Department

External Examiner: _____

Dr. Vaishali Patil
Principal

Date: _____

Acknowledgement

With immense pleasure, we are presenting this project report on “StrayCare: A Community Driven Platform for Animal Rescue and Adoption” as a part of the curriculum of B.E. Information Technology at International Institute of Information Technology, Hinjewadi, Pune.

It gives us the privilege to complete this report work under the valuable mentorship of Dr. Bhavana Kanawade. Her guidance, co-operation and encouragement has made a headway in the project. We are also extremely grateful to Dr. Vaishali Patil, Principal, and Dr. Jyoti Surve, Head of Department of Information Technology for providing all facilities and help for smooth progress of report work. We would also like to thank all the Staff Members of the Information Technology Department, Management, friends, and our family members, who have directly or indirectly guided and helped us for the preparation of this report and gave us unending support right from the stage the idea was conceived.

BIB04 Sanath Bhat

BIB21 Karttikey Chavali

BIB23 Hariom Khaladkar

BIB26 Rajat Kulkarni

Abstract

StrayCare is an innovative web-based platform developed to connect compassionate citizens with animal welfare organizations, creating a seamless channel for reporting and managing stray animal rescue cases. In many communities, people often witness injured or distressed animals but lack an easy and reliable way to request help. StrayCare fills this gap by offering a centralized system where citizens can quickly report incidents by uploading photos, sharing locations, and providing brief details about the situation.

The platform empowers NGOs with a dedicated dashboard to receive, track, and respond to these rescue requests efficiently, ensuring that every report is attended to in a timely manner. Administrators play a crucial role in maintaining platform integrity through a control panel that verifies NGO authenticity and manages overall operations.

Built with a React frontend for an interactive and intuitive user experience and a FastAPI backend for high-performance and secure API services, StrayCare ensures smooth communication between users and organizations. The system uses a SQLite database during development, providing simplicity and reliability for early-stage deployment.

Ultimately, StrayCare aims to foster a more compassionate, transparent, and efficient rescue network by leveraging modern web technologies. By bridging the gap between citizens and NGOs, it transforms community empathy into real-world action—making it easier to save and protect the lives of stray animals in need.

Chapter 1: Introduction to Project Topic

1.1 Overview

1.1.1 Project Overview

Across India's cities and towns, countless stray animals suffer from injuries, illness, and neglect, often in plain sight. While social media and local chat groups have become informal channels for reporting such incidents, these methods are fragmented, unreliable, and lack a structured response mechanism. Compassionate citizens who witness an animal in distress often feel helpless, unsure of whom to contact or how to provide effective aid. This communication gap between the public and animal welfare organizations leads to delayed rescues, wasted resources, and preventable suffering.

StrayCare is envisioned as a transformative digital solution to bridge this gap, democratizing the process of animal rescue. The platform acknowledges the public's desire to help but also recognizes their lack of direct connection to verified rescuers. By offering a mobile-first, easy-to-use, and location-aware platform, StrayCare empowers citizens to become effective first responders while enabling NGOs to manage rescue operations with unprecedented efficiency. Administrators ensure the integrity of the network, creating a trusted and coordinated ecosystem for animal welfare.

1.2 Brief Description

StrayCare is a comprehensive animal rescue and support portal developed using modern web technologies. It is built on a role-based access control system for three primary user types: Citizens, NGOs, and Administrators. Citizens can create geo-tagged case reports with photos, track the status of their reports, and view recovery stories. NGOs receive notifications for cases in their operational area, manage their caseload through a dedicated dashboard, and post progress updates. Administrators oversee the entire ecosystem by verifying new NGOs, managing users, and ensuring the platform's integrity.

The platform's core feature is its intelligent notification system, which routes new case reports

to the most relevant local NGOs based on location. By combining this efficient matching with a user-friendly design, StrayCare ensures that animals in distress receive timely and appropriate care.

1.3 Problem Definition

This project directly addresses the systemic inefficiencies in the current, informal stray animal rescue process. The key problems are:

- **Fragmented and Unreliable Reporting:** Citizens rely on scattered social media posts or unverified contact numbers, with no guarantee that their report will be seen by the right people in time.
- **Lack of Actionable Information for NGOs:** Rescuers are often inundated with duplicate reports, calls from outside their service area, or messages lacking critical details like a precise location or photo, leading to wasted time and resources.
- **No Feedback Loop for Citizens:** Individuals who report a case rarely receive updates on the animal's condition or outcome, leading to a sense of futility and discouraging future reporting.
- **Credibility and Verification Issues:** It is difficult for the public to distinguish between legitimate, registered NGOs and individual, unverified rescuers, creating a trust deficit.
- **Absence of a Centralized Knowledge Base:** Citizens who want to provide immediate first aid often lack access to safe and reliable instructions, potentially causing more harm than good.

1.4 Applying Software Engineering Approach

StrayCare is being developed using a structured software engineering methodology to ensure a robust, scalable, and maintainable product.

- **Requirements Engineering:** The development process began with a thorough analysis of the needs of the three key actors—Citizens, NGOs, and Admins—to define clear functional and non-functional requirements.

- **Architectural Design:** The system is designed using a three-tier architecture, separating the presentation (frontend), application logic (backend), and data layers. An API-first approach ensures that the system is modular and can be easily extended in the future (e.g., to support a native mobile app).
- **Technology Stack:** The chosen stack—React for the frontend and FastAPI (Python) for the backend—ensures consistency, maintainability, and high performance. React enables a dynamic and responsive user interface, while FastAPI provides a fast, secure, and modern API.
- **Quality Assurance:** A comprehensive testing strategy is planned, including unit testing for individual functions, integration testing for API endpoints, and User Acceptance Testing (UAT) with representative users to ensure the platform is intuitive and meets their needs.
- **Development Methodology:** The project follows an Agile, iterative development practice. By building the application in modular, incremental phases, we can incorporate feedback continuously and adapt to new requirements, ensuring the final product is well-aligned with its mission.

Chapter 2: Literature Survey

During the requirement gathering phase for the StrayCare platform, a comprehensive review of existing digital animal welfare solutions was conducted. This review analyzed numerous research papers to understand the current advancements, persistent challenges, and technological gaps in the field. The study of foundational systems, like the “Stray Animal Mobile App”, and more recent mobile applications, revealed a strong trend toward using mobile technology for basic citizen reporting. However, these studies also concluded that early prototypes often lacked the robust backend infrastructure needed for real-time tracking, multi-user coordination, and data validation. A systematic review of smart computing technologies in animal welfare confirmed that the field is maturing, yet a major gap identified in the literature is the fragmented nature of the current ecosystem.

2.1 Incident Reporting and Alert Systems

Current methods for reporting animals in distress are often disconnected from actual, verified rescue operations. While a significant body of research focuses on mobile reporting apps, these systems often function as “alert” mechanisms without a clear “case management” backend. This limitation was a key finding in the design of the “Stray Rescue Management System (SRMS)”, which noted that simple reporting is insufficient. More advanced systems are now integrating location-based technology and even AI for real-time assistance. The work by Rajwani et al. on using AI for real-time reporting, Sengan et al. on deep learning for roadway animal detection, and Chen, Liu & Liao on tracking stray animals, shows a clear path forward.

2.2 Case Management and Citizen Tracking

A major gap identified in the literature is the lack of transparency for the citizen after a report is made. Many organizations manage cases using internal-only tools, leaving the reporter with no updates. This “report-and-forget” dynamic discourages future community engagement. The work by Wu et al. identified this disconnect as a primary design challenge, emphasizing the need for a unified “rescue system” that formally connects the citizen’s report

to the NGO’s workflow. This is a core problem that StrayCare aims to solve by providing a citizen dashboard to track case status updates directly from the NGO.

2.3 Adoption and Rehoming Processes

The adoption modality is a popular field of research, but it is often treated as a completely separate problem from rescue. Many platforms focus on building better “storefronts” for adoption. More advanced systems are now using AI to improve this process. Jha et al. propose using AI to “find homes for the voiceless” through intelligent matching; Lu uses machine learning to predict adoption status; Torres et al. use content-based filtering for recommendations. However, all these platforms are largely disconnected from the animal’s rescue history, failing to provide a holistic view of its journey.

2.4 NGO Verification and Accountability

While many online directories list animal shelters, there is often a lack of rigorous, transparent verification processes to confirm the legitimacy and operational standards of listed NGOs. Crucially, the reviewed literature places very little emphasis on a structured, public-facing feedback and rating system. This makes it difficult for citizens to gauge the responsiveness and effectiveness of different organizations. StrayCare addresses this gap by implementing an Admin-led verification process and a contextual feedback module, where citizens can only review an NGO after a case has been verifiably handled by them.

2.5 Fragmented Systems

The overarching limitation identified is fragmentation. The tools for animal welfare are a scattered collection of disparate systems. A citizen might use the “Stray Watch Application” to report an animal, while an NGO uses a separate platform to list it. This lack of a unified system hinders data sharing and prevents a holistic view of animal welfare. StrayCare is designed to solve this by integrating all four key modalities—Reporting, Case Management, Adoption, and Accountability—into a single, cohesive platform.

Table 2.1: Comparative Analysis of Existing Systems

Paper Name	Algorithms &		Results &		
	Technology	Methodology	Outcomes	Limitations	Future Scope
Pet Adoption and Rescue Center	Proximity search, geolocation, AI image recognition. Web app (HTML/CSS/JS, Node.js, MongoDB, Google Maps API).	Client-server model with “Instagram-style” interface and 3-step adoption process.	Increased rescue reports, improved adoption flow, strong community engagement.	Needs higher user adoption; fragmented reporting issues.	Mobile app, AI pet matching, blockchain for transparent records.
CareForPaws	Personality & location-based matching. React Native mobile app with GPS.	Quantitative research with 15 users via surveys/interviews; evaluated using SUS.	All tests passed; SUS score 85 (Excellent); better than Facebook groups.	1MB image limit; only email verification.	Improve storage service; add multi-factor authentication.
‘Pet Home’ App	AI photo matching, proximity alerts. Concept using Big Data, AI, AR; prototype in Figma.	Human-Centred Design (Empathize-Prototype) with 14 participants.	High-fidelity prototype with 6 modules improving rescue system design.	No real-world testing; limited time/resources.	Real-world testing to validate effectiveness.

Paper Name	Algorithms &		Results &		
	Technology	Methodology	Outcomes	Limitations	Future Scope
Mobile App to Help Abandoned Pets	Business process logic for registration, adoption, blacklist. Database with 9 linked tables.	UML use case & class diagrams; admin-monitored adoption process.	Reduced abandon-ment via structured adoption and monitoring.	Lacked forum, no lost-pet reporting.	Add forum and lost-pet report feature.
PetHub	Standard user/data logic. Web app (HTML, CSS, PHP, MySQL).	Agile process with iterative development; 3 modules (Admin, Shelter, User).	Streamlined adoption & donations; improved efficiency and trust.	Limited transparency; weak filtering & post-adoption support.	Add mobile app, social media link, geolocation, real-time chat.

Looking at these existing systems, it's clear that while many projects have solved parts of the problem, they don't connect the full process. This creates a fragmented and inefficient "black box" where a citizen reports an animal, but their report gets disconnected from the NGO's management work, and the animal's rescue story is lost before it ever makes it to an adoption site. The main goal for our project, StrayCare, was to fix this exact fragmentation problem. We set out to build a single, unified platform that connects the entire journey from start to finish. To do this, we focused on four key objectives:

1. Build a reliable, standardized reporting system so citizens know their reports are being seen by the right people.
2. Create a transparent "My Reports" dashboard that closes the "report-and-forget" loop

and allows citizens to see the real-time status of their case.

3. Design an integrated workflow where an NGO can manage a “rescue case” and then seamlessly convert it into an “adoptable pet” on the same platform.
4. Establish trust and accountability with an Admin-led NGO verification process and a feedback system, so citizens can rate their experience with an NGO based on a completed case.

Chapter 3: Software Requirements Specification

3.1 Introduction

3.1.1 Purpose

The primary purpose of StrayCare is to bridge the gap between compassionate citizens who witness animals in distress and the verified Non-Governmental Organizations (NGOs) capable of providing rescue and medical aid. By addressing the fragmented, unreliable, and inefficient communication channels that currently dominate the stray animal rescue landscape, the system provides a technology-driven solution that routes actionable case reports to the most relevant local rescuers. At the same time, it simplifies case management for NGOs, reducing response times and optimizing resource allocation. Ultimately, StrayCare aims to promote social responsibility and animal welfare by connecting community members with a structured, transparent, and effective rescue ecosystem.

3.1.2 Intended Audience and Reading Suggestion

This document is intended for a broad range of stakeholders, including the software development team, UI/UX designers, quality assurance testers, and project managers who will use it as the primary guide for implementation and testing. Additionally, academic reviewers, animal welfare organizations, and potential municipal partners can use this document for evaluation and analysis. Readers are advised to begin with the project scope and system features to understand the platform's core capabilities, followed by the external interface and non-functional requirements to grasp the system's operational constraints and quality attributes.

3.1.3 Project Scope

StrayCare creates a comprehensive digital ecosystem connecting citizens with animal rescue services. It allows citizens to register and submit geo-tagged case reports with photographic evidence through a simplified, mobile-first interface. Verified NGOs receive real-time notifications for new cases in their designated operational areas and manage their entire workflow

through a dedicated dashboard—from case acceptance and rejection to posting live progress updates. The platform features a rule-based AI chatbot and a static knowledge base to provide immediate first-aid guidance. A robust administrative panel allows for the verification and management of all participating NGOs, ensuring the integrity and trustworthiness of the network.

3.1.4 Design and Implementation Constraints

The system is developed using a modern technology stack, with React for the frontend and FastAPI (Python) for the backend, ensuring a high-performance and maintainable architecture. As a web-based platform, it guarantees seamless access across mobile and desktop browsers. The backend is designed to be stateless, allowing for horizontal scalability to handle a growing number of concurrent users. Security is enforced through JWT-based authentication, role-based access control (RBAC), HTTPS encryption, and secure API endpoints. The user interface is designed with accessibility in mind, prioritizing clarity and ease of use for a non-technical audience.

3.1.5 Assumption and dependencies

The platform assumes that users (both Citizens and NGOs) have access to an internet-enabled device with a modern web browser and that GPS/location services are enabled for accurate case reporting. The system’s functionality is dependent on the availability of its cloud hosting infrastructure. The AI chatbot’s functionality, while currently rule-based, is designed to be extensible via third-party AI service APIs (e.g., Hugging Face). The system’s ability to send real-time alerts in future versions will depend on integration with push notification services like Firebase Cloud Messaging.

3.2 System Feature

3.2.1 User Authentication and Authorization

User authentication and authorization are critical for providing secure, role-based access for Citizens, NGOs, and Administrators.

- Users can register and log in via email and password. Passwords are never stored in plain text and are hashed using bcrypt.
- Authenticated sessions are managed using stateless JWT (JSON Web Tokens), which are sent with every protected API request.
- The system provides distinct portals & functionalities for each user role, ensuring that a Citizen cannot access NGO management functions & only an Admin can perform verification tasks.

3.2.2 Case Management and Workflow System

The core of the platform is its structured workflow for managing rescue cases from creation to resolution.

- **Case Reporting:** Citizens can create a case by submitting a description, a mandatory photo, and their GPS location. Each new case is created with a “Pending” status.
- **Case Assignment (Implicit):** All “Pending” cases are visible to all verified NGOs on their main dashboard.
- **Case Lifecycle Management:** An NGO can change a case’s status by “Accepting” or “Rejecting” it. Once accepted, the case is assigned to that NGO, and they can post progress updates. Citizens can track these updates in real-time on the case detail page.

3.3 External Interface Requirements

3.3.1 User Interface

StrayCare provides a mobile-first, responsive, and intuitive interface designed for ease of use.

- **Simplified Navigation:** The UI uses clear labels and a minimal number of clicks to perform critical actions like reporting a case.
- **Role-Based Views:** The navigation bar and available pages dynamically change based on the logged-in user’s role (Citizen, NGO, or Admin), ensuring a clean and relevant experience.
- **Feedback and Status Indicators:** The interface provides immediate feedback for

user actions, such as loading indicators, success messages, and clear error notifications.

3.3.2 Software Interface

- **Database:** The application uses SQLite for development and is designed for an easy migration to a production-grade relational database like PostgreSQL. The interaction is managed through the SQLAlchemy ORM.
- **Authentication:** Session management is handled via JWTs, providing secure and stateless access control.
- **API Communication:** The frontend (client) and backend (server) communicate exclusively through a RESTful API, using JSON for standardized data exchange.

3.3.4 Communication Interface

- **REST API:** The application relies on HTTPS-secured RESTful APIs for all client-server communication.
- **AI Chatbot:** The backend exposes a `/chatbot/query` endpoint that processes user input against a set of pre-defined rules in `chatbot_logic.py`, requiring no external API calls for this feature.
- **Error Handling:** The API uses standard HTTP status codes (200, 201, 401, 404, 422, etc.) to communicate the outcome of requests.

3.4 Non-Functional Requirements

3.4.1 Performance Requirements

StrayCare is engineered for fast and reliable performance.

- **API Response Time:** The backend is designed to maintain an average API response time of under 500 milliseconds for all standard data retrieval operations.
- **Page Load Time:** The React frontend is optimized for fast initial rendering, with a target First Contentful Paint (FCP) of under 2 seconds on a standard 3G connection.
- **Database Performance:** Database queries are optimized through proper indexing on key columns (e.g., user emails, case statuses) to ensure efficient data retrieval.

3.4.2 Safety Requirements

- **Data Integrity:** All user inputs are validated on both the client and server sides using Pydantic schemas to prevent invalid or malicious data from entering the database.
- **Database Backups:** Automated, regular database backups will be configured in a production environment to safeguard against data loss or corruption.
- **Graceful Error Handling:** The system is designed to handle unexpected errors gracefully, providing informative messages to the user without crashing or exposing sensitive system information.

3.4.3 Security Requirements

Security is integral to maintaining user trust and protecting sensitive information.

- **Authentication:** The platform implements JWT-based authentication with role-based access control (RBAC), ensuring that users can only access functionalities appropriate to their roles.
- **Data Encryption:** All data transfers between the client and server occur over HTTPS. All passwords are irreversibly stored using bcrypt hashing.
- **Vulnerability Prevention:** The system includes measures to prevent common web vulnerabilities, such as input sanitization and protection against SQL injection (via the ORM) and cross-site scripting (XSS) (via React's default data binding).

3.4.4 Software Quality Attributes

- **Maintainability:** The codebase is highly maintainable, following best practices such as a modular architecture (React components, FastAPI routers), separation of concerns (e.g., crud.py for database logic), and clear documentation.
- **Scalability:** The backend is stateless, allowing for horizontal scaling by deploying multiple instances behind a load balancer in a production environment.
- **Reusability:** The frontend is built with reusable React components (e.g., forms, cards, modals), which accelerates future development.
- **Portability:** As a web application, the platform is portable and functions seamlessly

across all modern web browsers (Chrome, Firefox, Safari, Edge) on desktop and mobile operating systems.

3.5 Analysis Model

The analysis model for StrayCare focuses on evaluating the effectiveness and efficiency of the rescue workflow. Unlike a recommendation engine, the core success of this platform is measured by its real-world impact on rescue operations.

- **Key Performance Indicators (KPIs):** The system is designed to facilitate the tracking of several critical metrics:
 - **Average Response Time:** The time elapsed from a case being reported to an NGO accepting it.
 - **Case Resolution Rate:** The percentage of reported cases that are eventually marked as resolved or closed.
 - **Geographic Hotspot Analysis:** Identifying areas with a high frequency of reported cases to help NGOs allocate resources more effectively.
 - **NGO Activity & Performance:** Tracking the number of cases accepted and updated by each NGO to identify the most active and effective organizations in the network.
- **Evaluation:** This data-driven analysis will validate the platform's ability to reduce rescue times, increase the number of successful interventions, and provide a clear picture of animal welfare needs across different localities, reinforcing StrayCare's mission.

Chapter 4: System Design

4.1 System Architecture

The architecture of StrayCare is designed as a modern, multi-layered client-server system to ensure maintainability, clear separation of concerns, and future scalability. It consists of a Presentation Layer (the React Frontend), a Logic Layer (the FastAPI Application), and a Data & Storage Layer (the SQLite database and static file storage).

1. The User — This is the Actor (the Citizen, NGO, or Admin) who interacts with the system using their web browser.

2. The Browser (Client-Side) — This entire box represents what runs on the user's computer.

- **React Frontend:** This is your entire user interface (UI). It's responsible for drawing all the pages, buttons, and forms.
- **Pages (Views):** These are the main “screens” of your app, like the LandingPage, AdoptionPage, or NGODashboard.
- **Components (UI):** These are the reusable building blocks that make up the pages, such as PetCard, CaseCard, or the login form.
- **API Client (Axios):** This is the “messenger.” When you click a button (e.g., “Submit Report”), this client is responsible for bundling up your data and sending it to the backend.

3. The Connection (HTTP Requests/Responses) — This is the communication channel between the frontend and the backend.

- **HTTP API Requests (JSON):** The frontend sends data (like a new user's email and password) to the backend in a standard JSON format.
- **HTTP API Responses (JSON):** The backend sends data (like a list of pets or a success message) back to the frontend, also in JSON format.

4. The Backend Server (Server-Side) — This is your “brain” and “filing cabinet” that runs on a server. It handles all the logic, security, and data storage.

- **FastAPI Application:** This is the main framework that organizes your backend.
 - **API Endpoints (`main.py`):** This is the “front door” of your backend. It defines all the URLs (like `/pets`, `/report`, `/ngo/token`) that the frontend is allowed to call.
 - **Security (`security.py`):** This is the “bouncer.” It checks the user’s login token on protected routes to make sure they are authorized to perform an action (e.g., only an Admin can access `/admin/feedback`).
 - **Data Schemas (`schemas.py`):** This is the “rulebook” or “validator.” It ensures that data sent from the frontend is in the correct format (e.g., is the email field actually an email? Is the rating a number between 1 and 5?).
 - **Business Logic (`crud.py`):** This is the “engine room.” The endpoints call functions in this file to do the actual work—like creating a user, fetching all cases for an NGO, or saving a new pet to the database.
 - **Database Models (`models.py`):** This is the “blueprint” for your database. It defines the structure of your tables (like `User`, `Case`, `Pet`) and their relationships, allowing the Business Logic to interact with the database in a clean, object-oriented way.
- **Static Files (`uploads/`):** This is a simple folder on the server. When a user uploads a photo (for a case or a pet), the Business Logic saves that image file here. The database only stores the text path to this file (e.g., `uploads/pet_123.jpg`).
- **SQLite Database (`straycare.db`):** This is your main database file. It’s the “filing cabinet” that stores all your text and number-based data—user info, case descriptions, adoption requests, feedback comments, etc.

4.2 UML Diagrams

UML (Unified Modeling Language) diagrams are used to visually represent the system’s structure and behavior.

4.2.1 Module 1: Case Reporting & Management

Use Case Diagram (Fig 4.2) illustrates the primary rescue workflow. It defines the external actors, “Citizen” and “NGO,” and their interactions with the system. The Citizen initiates the process by reporting and tracking a case, while the NGO is responsible for the internal management, which includes the actions of accepting, rejecting, or updating the case.

Activity Diagram (Fig 4.3) shows the complete lifecycle and flow of a single rescue case. The activity begins with the “Citizen Submits Report,” which creates a “Pending” case. From there, it flows into a decision node where an NGO’s action (Accept or Reject) determines the case’s transition to either the “Accepted” or “Rejected” state, with the “Accepted” path including further activities like providing updates and eventual resolution.

Sequence Diagram (Fig 4.4) details the step-by-step technical interactions for the “Report and Accept” workflow. It shows the exact sequence of messages, starting with the Citizen’s POST /report request, followed by the NGO’s GET /ngo/me/cases to retrieve the data, and concluding with the NGO’s PUT /case/{id}/accept call, demonstrating the data flow between the frontend, backend, and database.

4.2.2 Module 2: Pet Adoption Workflow

Use Case Diagram (Fig 4.5) details the two sides of the adoption module. It shows the “NGO” actor’s responsibility to “List Pet for Adoption” and “Manage Adoption Requests” (which includes approval or rejection). It also shows the “Citizen” actor’s complementary actions: “Browse Adoptable Pets” and “Submit Adoption Request.”

Activity Diagram (Fig 4.6) illustrates the end-to-end lifecycle of a pet on the platform. The activity begins with the “NGO Submits ‘List Pet’ Form,” which places the pet in an “Available” state. The workflow then enters a loop where citizens can submit requests, which are reviewed by the NGO. An “Approve” action by the NGO breaks the loop, transitioning the pet’s status to “Adopted” and finalizing the process.

Sequence Diagram (Fig 4.7) shows the technical implementation of the adoption workflow. It is divided into two distinct sequences: first, the NGO lists a pet by sending a POST

/ngo/pets request, which creates a new pet in the database. Second, a Citizen retrieves this data via a GET /pets request and then submits an application using POST /adoption-requests.

4.2.3 Module 3: NGO Verification & Feedback

Use Case Diagram (Fig 4.8) focuses on the platform’s accountability and trust mechanisms. It shows the “Admin” actor’s critical role in “Verify NGOs” and “View All Feedback.” It also illustrates the feedback loop: the “Citizen” can “Leave Feedback,” and the “NGO” can “View Own Feedback,” creating a transparent review system.

Activity Diagram (Fig 4.9) shows two parallel workflows. The “NGO Verification” partition details the process of an Admin reviewing an application, leading to either a “Verified” or “Denied” state. The “Feedback Lifecycle” partition shows the simple flow starting with a citizen’s submission, which then populates the database for both the NGO and Admin to view.

Sequence Diagram (Fig 4.10) details the feedback data flow from all three perspectives. It begins with the Citizen submitting a POST /feedback request. It then shows how the NGO retrieves their specific, aggregated feedback summary via GET /ngo-feedback, and finally how the Admin retrieves a complete list of all feedback on the platform using the protected GET /admin/feedback endpoint.

4.2.4 Class Diagram (Fig 4.11)

Illustrates the static data structure of the StrayCare system, corresponding to the database models. It defines the “classes” (or tables) like User, Case, Pet, and Feedback, detailing the attributes (columns) and data types for each. Most importantly, it maps the relationships (cardinality) between them, such as the one-to-many relationship between a User and the Cases they report, or between an NGO and the Pets they list.

4.2.5 Object Diagram (Fig 4.12)

Shows a specific, real-world snapshot of the system at a moment in time. Unlike the Class Diagram, which is a blueprint, this diagram uses named instances (objects) to demonstrate a scenario. Here, it shows “jane_doe” (a User object) who has reported “case_101” (a Case object). This case has been accepted by “test_ngo” (an NGO object), which in turn has listed “pet_buddy” (a Pet object).

4.2.6 Component Diagram (Fig 4.13)

Provides a high-level view of the system’s software architecture, breaking it into its main logical components. It shows the React Frontend (which runs in the user’s browser) and the StrayCare Backend System. It illustrates that the frontend “uses” the REST API interface, which is “implemented” by the FastAPI Application. It also clearly shows the backend’s dependencies on the SQLite DB and the File Storage.

4.2.7 Deployment Diagram (Fig 4.14)

Shows the physical or virtual hardware (nodes) on which the software components are deployed. It illustrates that the User interacts with a Client Device (like a phone or laptop), which runs the React SPA inside a Web Browser. This client communicates via HTTP with a Cloud Server, which hosts the FastAPI Application, the Database, and the File System for image uploads.

Chapter 5: Literature Survey on “Rule-Based Approach in Natural Language Processing (NLP)”

5.1 Introduction

Natural Language Processing (NLP) enables systems to understand and interpret human language. In emergency response platforms like StrayCare, understanding the urgency of a reported text is critical. Rule-based approaches in NLP rely on predefined linguistic rules and lexicons rather than statistical models, making them highly deterministic, fast, and reliable for specialized domains like medical triage.

5.2 Previous Research

Historically, automated dispatch systems utilized simple keyword matching. Recent research has shifted towards hybrid models where rule-based systems function as a primary triage layer, followed by deep learning models for deeper context extraction. Studies on urban emergency response indicate that rule-based NLP provides the lowest latency, which is the most critical metric in trauma cases.

5.3 State of Art Algorithms

While Large Language Models (LLMs) like GPT-4 are current state-of-the-art, their reliance on API calls introduces latency, privacy concerns, and recurring costs. For focused triage execution, handcrafted lexicons weighted by domain experts (e.g., veterinarians) often outperform generalized models in zero-shot environments. The StrayCare platform deliberately avoids paid external AI APIs, implementing all intelligence locally on the server or in the browser.

5.4 Handcrafted Rules in Rule-Based Sentiment & Severity Analysis

In StrayCare, we implemented a custom Rule-Based NLP engine in `algorithms.py`. The algorithm tokenizes the citizen’s case description and applies a weighted scoring system based

on predefined arrays:

- **Critical (+100 pts):** Keywords like *bleeding, unconscious, hit by car, paralyzed, seizure, not breathing*.
- **High (+60 pts):** Keywords like *injured, poisoned, trapped, bitten, attacked, broken leg*.
- **Moderate (+30 pts):** Keywords like *limping, malnourished, abandoned, lost, shivering*.

The final score determines the severity label assigned:

- **Score ≥ 100 :** Label = “Critical” (Red badge on NGO dashboard)
- **Score ≥ 60 :** Label = “High” (Orange badge)
- **Score ≥ 30 :** Label = “Moderate” (Yellow badge)
- **Score < 30 :** Label = “Low” (Green badge)

This deterministic approach allows the system to instantly classify the case and sort it with a colored priority badge in the NGO’s dashboard without relying on external APIs.

Chapter 6: Machine Learning Concepts and Implementation

6.1 Methodology and Algorithms

StrayCare employs three distinct mathematical and ML mechanisms natively:

1. K-Means Clustering (Unsupervised ML) — Zone Red Hotspot Map

Used to power the “Zone Red Hotspot Map”. The algorithm processes the GPS coordinates (latitude/longitude) of all reported cases. It randomly initializes K centroids and iteratively assigns spatial points to the nearest centroid using the Euclidean distance formula until convergence. The result maps high-risk geographic danger zones for NGOs to deploy proactive resources. This runs entirely in pure Python on the backend with no external ML library required.

2. MobileNet V2 via TensorFlow.js (Supervised CNN) — Animal Species Recognition

A Convolutional Neural Network deployed entirely in the client’s browser via TensorFlow.js. It classifies uploaded pet or case photos across 1000 categories, allowing the application to auto-detect species (dog, cat, bird, etc.) without server roundtrips. This protects user privacy as images never leave the user’s device.

3. Haversine Distance Formula — Smart NGO Dispatch

A geospatial algorithm used to calculate the great-circle distance between the reported emergency coordinates and all registered NGOs. The dispatch scoring matrix then ranks NGOs by combining: Distance (primary), Current Active Caseload (secondary), and Animal Specialization (tertiary). The final ranked list is presented to the Admin for one-click assignment.

6.2 Confusion Matrix for Decision Tree Classifier

For supervised classification modules like species recognition, performance is evaluated using a Confusion Matrix determining True Positives (TP), True Negatives (TN), False Positives (FP), and False Negatives (FN). The key evaluation metrics are:

- **Precision** = $TP / (TP + FP)$ — Of all animals labeled as “Dog”, how many were actually dogs.
- **Recall** = $TP / (TP + FN)$ — Of all actual dogs, how many were correctly identified.
- **F1-Score** = $2 * (Precision * Recall) / (Precision + Recall)$ — Harmonic mean balancing both metrics.

These metrics are prioritized to ensure the UI confidence badges (e.g., “85% Confidence: Dog”) accurately reflect the model’s certainty. The MobileNet V2 model achieves approximately 71.8% top-1 accuracy on the ImageNet benchmark.

6.3 Enhancements and Future Work

Future versions of the StrayCare ML pipeline will:

- Transition the rule-based NLP triaging to a fine-tuned lightweight transformer model (e.g., BERT-tiny or DistilBERT) trained specifically on animal rescue descriptions.
- Integrate time-series forecasting (ARIMA/Prophet) with the K-Means clusters to predict future accident hotspots based on seasonal trends (e.g., monsoon impact on stray animals).
- Replace MobileNet with a custom-trained species classifier fine-tuned on Indian street animal images for higher domain-specific accuracy.

Chapter 7: Software Testing and Quality Assurance

7.1 Introduction to Testing

To ensure the StrayCare platform is highly robust and capable of handling critical emergency data securely, a comprehensive testing strategy was implemented. This strategy combines Automated Testing for backend API stability and algorithmic accuracy, with Manual Testing for user experience (UX) validation and edge-case discovery.

7.2 Automated Testing

Automated testing scripts execute continuously to verify system integrity without human intervention.

1. Unit Testing (Backend Algorithms)

Utilizing standard Python testing practices, individual functions in `algorithms.py` were tested in isolation. Key test cases included:

- `haversine_distance(lat1, lon1, lat2, lon2)` — Tested with known city coordinates to verify distance accuracy within 0.1 km tolerance.
- `calculate_severity_score(description)` — Tested with pre-classified sentences to verify correct label assignment for Critical, High, Moderate, and Low cases.
- `cluster_case_hotspots(cases, k)` — Tested with sample GPS coordinate arrays to verify centroid convergence.

2. API Endpoint Testing (Postman / FastAPI Swagger)

Automated request-response cycles were tested for all major routes:

Endpoint	Method	Expected Status	Test Scenario
<code>/users/register</code>	POST	201 Created	Valid user registration
<code>/users/login</code>	POST	200 OK + JWT	Correct credentials
<code>/users/login</code>	POST	401 Unauthorized	Wrong password
<code>/report</code>	POST	201 Created	Valid case with image

Endpoint	Method	Expected Status	Test Scenario
/admin/hotspots	GET	200 OK	Admin token present
/admin/hotspots	GET	401 Unauthorized	No token provided
/ngo/me/cases	GET	200 OK	NGO token present

3. Database Schema Migration Testing

Each new column added to the SQLite schema (e.g., `severity_score`, `severity_label`) was tested by verifying: (a) existing records receive default values, (b) new records correctly populate the column, and (c) the backend starts without schema mismatch errors.

4. Frontend Hook Testing

The `useAnimalRecognition` hook was tested by simulating image uploads and verifying state transitions through: `idle` -> `loading` -> `analyzing` -> `done` with the correct confidence percentage and species label returned.

7.3 Manual Testing

Manual testing evaluated the application from the perspective of our three distinct user personas (Citizens, NGOs, Admins).

1. User Acceptance Testing (UAT)

Volunteers navigated the system using realistic scenarios. Citizens simulated uploading photos with variable lighting conditions (bright sunlight, night, blurry) to test the resilience of the AI species recognition. Testers verified that the TensorFlow.js model gracefully returned a “Low confidence” result rather than crashing on ambiguous images.

2. Role-Based Access Control (RBAC) Testing

Systematic security checks were performed to ensure:

- A logged-in Citizen navigating manually to `/ngo/dashboard` was redirected to the login page.

- A logged-in NGO attempting to call `GET /admin/feedback` received a 401 `Unauthorized` response.
- An unverified NGO account received a 403 `Forbidden` error when attempting to accept cases.

3. End-to-End (E2E) Rescue Lifecycle Testing

Evaluators walked through the complete lifecycle of a rescue operation:

1. Citizen opens Report Form, enables GPS location (verified accurate on Chrome, Edge, and Firefox).
2. Citizen uploads a photo — TensorFlow.js returns “Dog detected (82% confidence)” and auto-fills description.
3. Citizen submits — NLP algorithm triggers, assigns “High” severity, orange badge appears.
4. Admin opens Smart Dispatch — Haversine-ranked list of 3 nearest NGOs is displayed.
5. Admin assigns Case to NGO-1.
6. NGO-1 logs in, sees the orange-badged priority case, clicks “Accept.”
7. NGO posts a progress update: “Animal picked up, en route to clinic.”
8. Citizen sees the update appear on My Reports page in real-time.
9. NGO marks case “Resolved” and lists the recovered animal as an adoptable pet.
10. Admin views Zone Red hotspot map — the reported coordinates appear in the updated cluster.

4. Payment Gateway Verification (Razorpay Sandbox)

Manual testing in Razorpay’s Sandbox environment was conducted to simulate:

- Successful UPI transactions — verified donation status updated to “Completed” in DB.
- Failed transactions — verified webhook correctly set status to “Failed” without corrupting existing records.
- Duplicate webhook calls — verified idempotency checks prevented double-counting.

5. Cross-Browser & Responsive UI Testing

Glassmorphism styling, TensorFlow.js loading, and mobile GPS capture were tested on:

- Chrome (Desktop & Android)
- Microsoft Edge (Desktop)
- Mobile Safari (iOS)
- Firefox (Desktop)

All pages passed visual layout checks at 320px (mobile), 768px (tablet), and 1280px (desktop) viewport widths.

Chapter 8: Applications of Rule Based Approach in NLP systems

The combination of Rule-Based NLP and local Machine Learning within StrayCare demonstrates how modular AI can be applied to civic platforms efficiently:

1. Emergency Prioritization Instantly filtering high-trauma cases for immediate action. A citizen’s plain-language description like “dog lying in road, bleeding from head” is automatically scored as Critical and elevated to the top of every NGO’s dashboard queue, without any manual review.

2. Resource Allocation Using NLP severity data combined with Haversine distance calculations to prevent NGO burnout and optimize vehicle routing. A “Low” severity case in a far location will not displace a “Critical” nearby case in the dispatch queue.

3. Proactive Civic Planning Utilizing K-Means hotspot clustering to advise municipalities on where to build animal shelters, install warning signs, or deploy mobile veterinary units based on statistically significant cluster centroids.

4. Adoption Optimization The lifestyle-matching algorithm drastically lowers the return rate of adopted animals by ensuring a mathematical compatibility score between the human’s living conditions (apartment size, hours away from home, activity level) and the pet’s temperament and energy requirements.

5. Data-Driven NGO Performance Monitoring Admin analytics dashboards powered by aggregated case data allow identification of which NGOs have the fastest average response times, highest case acceptance rates, and best citizen feedback scores—enabling evidence-based resource grants and recognition.

Chapter 9: Methodology and Results

9.1 Technology Details used in the project

The frontend of StrayCare is developed using modern web technologies to ensure a highly interactive, responsive, and maintainable user experience.

- The core library is **React.js (version 18.x)**, a component-based UI library that facilitates the creation of a dynamic and fast Single-Page Application (SPA).
- State management is handled efficiently using React's built-in hooks, primarily **useState** and **useEffect**, which are ideal for managing component-level and local application state.
- Styling is implemented using **CSS Modules**, which locally scopes class names to each component. This prevents global style conflicts, promotes a clean and modular design, and ensures visual consistency across the platform.
- **React Router** is employed for client-side navigation, enabling smooth, declarative transitions between different pages (e.g., /my-reports, /admin/dashboard) without requiring a full page reload.
- Data exchange between the frontend and the backend is managed using **Axios**, a promise-based HTTP client that ensures reliable and structured API communication.

The backend is built using a robust and high-performance Python-based stack designed for scalability and security.

- The **FastAPI** framework is utilized to create RESTful APIs, chosen for its exceptional speed, asynchronous capabilities, and automatic data validation powered by Pydantic.
- The application is run on a **Uvicorn** server, a lightning-fast ASGI (Asynchronous Server Gateway Interface) implementation.
- For data storage and retrieval, **SQLite** is employed as the development database, with **SQLAlchemy** serving as the Object-Relational Mapper (ORM). SQLAlchemy simplifies and secures database operations by allowing developers to interact with tables using Python objects.
- The backend incorporates **JSON Web Tokens (JWT)** for stateless authentication.

Passlib is used for securely hashing passwords with the bcrypt algorithm, while **Python-JOSE** handles the encoding and decoding of JWTs.

9.2 Procedure of the project

The development of StrayCare follows an Agile, iterative methodology, emphasizing modular development and continuous improvement to ensure the final product is robust and well-aligned with user needs. The process is broken down into distinct phases:

- **1. Planning & Requirements Engineering:** This initial phase involved a comprehensive analysis of the needs of the three primary user roles: Citizens, NGOs, and Administrators. This led to a detailed Software Requirements Specification (SRS) defining clear functional and non-functional requirements.
- **2. Design:** During the design phase, the UI/UX was conceptualized to be simple and intuitive. The database schema was structured in `models.py`, defining the tables and their relationships. A detailed API specification was laid out, defining all necessary endpoints, their request methods, and the expected data payloads.
- **3. Implementation:** This phase involved the iterative coding of the application's modules. Backend development focused on building the API endpoints, security functions, and database logic, while frontend development focused on creating the React components and user interfaces.
- **4. Testing:** A multi-layered testing strategy is employed to ensure quality. This includes manual testing of user flows, API testing using browser developer tools, and planned unit/integration testing to verify the correctness of individual functions and components.
- **5. Deployment & Maintenance:** The final phase involves deploying the application to a cloud hosting environment. Post-deployment, the platform will be continuously monitored for performance and stability.

The project follows a strategic implementation approach to maximize efficiency and quality.

- A **database-first methodology** ensures that the data models (`models.py`) are designed early, establishing a solid and reliable foundation for all backend operations.

- An **API-first development approach** allows the backend API to be built and tested independently, enabling parallel frontend development against a clear and stable contract. This is supported by FastAPI’s automatic interactive documentation.
- The frontend is constructed using a **component-based design**, where the UI is broken down into small, reusable React components (e.g., CaseCard, Login form). This approach ensures a consistent user experience and simplifies future updates.
- The backend follows a **Repository Pattern** by separating database logic (`crud.py`) from business logic (`main.py`), making the code cleaner, more modular, and easier to test and maintain.

9.3 Results

9.3.1 Citizen Module

Figure 9.1 — Citizen Dashboard

Figure 9.1 shows the Citizen Dashboard Interface, which serves as the main navigation hub for a logged-in user. It provides two clear, card-based options: “View My Reported Cases” to track existing reports and “Report a New Case” to submit a new one, streamlining the user’s primary actions.

Figure 9.2 — Report a Case Form

Figure 9.2 shows the Report a Case Interface, where a citizen can submit a new incident. It includes form fields for a description, an image upload, and a “Get Location” button, ensuring that all necessary data for an NGO to respond is captured in a standardized format. The integrated TensorFlow.js animal recognition module automatically detects the species from the uploaded photo and pre-fills the description field.

Figure 9.3 — My Reports Page

Figure 9.3 shows the My Reports Page, which displays a list of all cases previously submitted by the citizen. Each case item shows the status (e.g., “Pending,” “Accepted”), a summary, and provides a “View Details” button to track the case’s progress.

Figure 9.4 — Case Detail & Feedback View

Figure 9.4 shows the Case Detail Interface from the citizen’s perspective, which “closes the loop” on a report. It displays the NGO’s “Update Timeline” for the case and, once the case is resolved, presents the user with a “Leave Feedback” button to rate the NGO’s performance.

9.3.2 NGO Module

Figure 9.5 — NGO Dashboard (Case Management)

Figure 9.5 shows the main NGO Dashboard, which functions as a command center for case management. It displays all incoming “Pending” reports in a card format, allowing the NGO to efficiently triage new cases and take immediate action by clicking “Accept” or “Reject.” Cases are sorted by severity score and display color-coded priority badges (Red = Critical, Orange = High, Yellow = Moderate, Green = Low) powered by the NLP algorithm.

Figure 9.6 — List a Pet for Adoption Form

Figure 9.6 shows the List a Pet for Adoption Interface, a modal form used by NGOs to create new adoption listings. It includes fields for the pet’s name, species, breed, age, and an image upload, linking the rescue workflow directly to the public adoption gallery.

Figure 9.7 — Adoption Request Management

Figure 9.7 shows the Adoption Request Management Interface for NGOs. This page displays all submitted applications in a clear table, allowing the NGO to “View Details” on an applicant and then “Approve” or “Reject” the request, thereby managing the entire adoption pipeline.

Figure 9.8 — NGO Feedback Summary

Figure 9.8 shows the NGO Feedback Summary page, which displays the NGO’s own performance metrics. It provides an average star rating based on all submitted reviews and lists the individual comments from citizens, promoting transparency and accountability.

9.3.3 Admin Module

Figure 9.9 — Admin Dashboard (NGO Verification)

Figure 9.9 shows the Admin Dashboard, which is focused on platform integrity and trust. This section displays a list of all NGOs awaiting verification, allowing the admin to view their uploaded documents and “Verify” them, which grants them access to the platform.

Figure 9.10 — Admin Feedback Oversight

Figure 9.10 shows the Admin Feedback Oversight Interface, where an administrator can view all feedback submitted across the entire platform. This provides a high-level view of NGO performance and allows the admin to investigate any potential issues or disputes.

Chapter 10: Conclusion

In addressing the complex challenge of stray animal welfare, the StrayCare project has culminated in the creation of a comprehensive digital ecosystem designed to foster collaboration and positive action. This platform transcends simple functionality; it represents a vital connection point for compassionate citizens, dedicated animal welfare organizations, and animals desperately in need. By integrating intuitive reporting tools, accessible adoption pathways, and mechanisms for feedback and accountability, StrayCare provides a structured, technology-driven solution to enhance rescue coordination and promote responsible pet ownership.

Furthermore, by integrating advanced intelligence—ranging from NLP-based severity triaging to K-Means spatial clustering and in-browser TensorFlow.js image recognition—StrayCare validates its technical efficacy while maintaining a strict offline-first, privacy-preserving AI philosophy. The rigorous manual and automated testing protocols documented in Chapter 7 guarantee a secure, user-friendly, and bug-free experience across all user roles and devices.

Ultimately, this project serves as a testament to how thoughtful design and development can empower communities to work together more effectively, creating a tangible impact on the lives of vulnerable animals and building a more humane environment for all. The foundations laid by StrayCare offer a significant opportunity for continued growth and refinement, promising a brighter future for animal welfare initiatives.

Chapter 11: References

1. R. Kolandaisamy, K. Subaramaniam, I. Kolandaisamy, and L. S. Li, *Stray Animal Mobile App*, UCSI University, Kuala Lumpur, Malaysia, 2016.
2. A. Abdulkarim, M. A. K. B. G. Khan, and E. Aklilu, “Stray Animal Population Control: Methods, Public Health Concern, Ethics, and Animal Welfare Issues,” *World Veterinary Journal*, vol. 11, no. 3, pp. 319–326, 2021.
3. L. L. A. Walter, *StandForPaw: Animal Rescue and Pet Adoption Mobile Application*, Universiti Teknologi PETRONAS, Malaysia, 2021.
4. C. L. Yi and C. S. C. Dalim, “Development of Mobile Application for Animal Conservation: Animal Rescue,” *Applied Information Technology and Computer Science*, vol. 4, no. 2, pp. 430–449, 2023.
5. Y. H. Jean and N. Wahid, “FurRescue: A Mobile Application for Pet and Stray Animal Locator with Geo-Fencing and AI Breed Detection,” *Applied Information Technology and Computer Science*, vol. 6, no. 1, pp. 1–20, 2025.
6. Wu et al., 2022 - “Widespread of Stray Animals: Design a Technological Solution to Help Build a Rescue System for Stray Animals.”
7. Neha Jha et al., 2024 - “Using Machine Learning and AI to Find Homes for the Voiceless.”
8. Sengan et al., 2021 - “Real-Time Automatic Investigation of Indian Roadway Animals by 3D Reconstruction Detection Using Deep Learning for R-3D-YOLOv3.”
9. Rahajeng et al., 2024 - “Mobile Application to Help Abandoned Pets.”
10. Blancaflor et al., 2023 - “CareForPaws: A Mobile Application for Pet Adoption and Other Services with Location-Based Technology.”
11. Bricman & Kozuh, 2024 - “User Experience and Interface Design for a Digital Pet Adoption Platform (PetScout).”
12. Jay Prajapati et al., 2025 - “Pet Adoption and Rescue Center.”
13. Mahak Rajwani et al., 2025 - “Enhancing Animal Rescue Efficiency Using AI for Real-Time Reporting and Assistance.”
14. Honglin Lu, 2025 - “Pet Adoption Status Prediction Based on Multiple Machine

- Learning Models.”
15. Kalaivani A/P Subramaniam & Hazinah Kutty Mammi, 2025 - “Stray Rescue Management System (SRMS).”
 16. Torres et al., 2024 - “A Web-Based Animal Adoption and Rescue System that uses Content-Based Filtering Algorithm.”
 17. Jukan, Masip-Bruin & Amla, 2016 - “Smart Computing and Sensing Technologies for Animal Welfare: A Systematic Review.”
 18. FastAPI Documentation: <https://fastapi.tiangolo.com/>
 19. TensorFlow.js Models & MobileNet: <https://www.tensorflow.org/js>

Chapter 12: Project Plan 1.0

Activity Planned / Module(s)		
Week	Covered	Status
Week 1	Requirement Gathering & Analysis – Research existing systems. Identify actors (Citizen, NGO, Admin) and define core modules (Reporting, Adoption, Feedback).	Completed
Week 2	System Design & Database – Design system architecture (React, FastAPI). Create ERD, DFDs, and Use Case diagrams. Implement database schema (models.py).	Completed
Week 3	Backend Core & Auth Module – Set up FastAPI backend. Implement security (security.py) and all authentication endpoints (/token, /ngo/token, /register).	Completed
Week 4	Frontend Core & Auth Module – Set up React frontend. Implement routing (App.js) and all user/NGO login and registration pages.	Completed

	Activity Planned / Module(s)	
Week	Covered	Status
Week 5	Case Reporting Module – Build backend (POST /report) and frontend (ReportCase.jsx) for case submission. Build citizen’s MyReports.jsx page.	Completed
Week 6	NGO Case Management Module – Build backend endpoints (GET /ngo/me/cases, Accept/Reject case). Implement NGODashboard.jsx for NGOs to manage cases.	Completed
Week 7	Case Detail & Update Module – Build backend (GET /cases/{id}, POST /cases/{id}/updates). Implement CaseDetail.jsx page for citizens and “Add Update” modal for NGOs.	Completed
Week 8	Pet Adoption Module (Backend) – Implement all adoption endpoints: POST /ngo/pets (List Pet), GET /pets (View Gallery), POST /adoption-requests (Apply).	Completed

Week	Activity Planned / Module(s)	
	Covered	Status
Week 9	Pet Adoption Module (Frontend) – Build AdoptionPage.jsx gallery, AdoptionModal (for citizens), and AddPetForm.jsx (for NGOs).	Completed
Week 10	Feedback & Admin Module – Build all Feedback endpoints and Admin endpoints. Implement AdminDashboard.jsx, LeaveFeedbackModal.jsx, and NGOFeedback.jsx.	Completed
Week 11	AI & Algorithm Module – Implement Rule-Based NLP Severity Scoring, Haversine Smart Dispatch, K-Means Hotspot Clustering, and TensorFlow.js Animal Recognition.	Completed
Week 12	Integration & Testing – Final end-to-end integration testing. Automated API tests, Manual E2E flow testing, Cross-browser UI validation.	Completed

	Activity Planned / Module(s)	
Week	Covered	Status
Week 13	Final Documentation – Write the complete research paper, project report, and prepare the final presentation/demo.	Completed